

Homework4 参考答案

本题是用 gdb 和 valgrind (memcheck) 观察各种内存错误的实验题，下面给出每个小问 的现象与解释。假设在 Linux + glibc 下用 gcc 编译。

Problem 1

1. null.c：创建一个 int 指针置为 NULL 再解引用写入。

```
#include <stdio.h>
int main() {
    int *p = NULL;
    *p = 0;          /* 解引用空指针 */
    return 0;
}
```

运行时访问虚拟地址 0，触发 page fault 且该地址不可访问，内核给进程发 SIGSEGV，程序崩溃：Segmentation fault (core dumped)。

2. 加 -g 编译后在 gdb 里 run：gdb 捕获到 SIGSEGV，并指出出错的源代码行，类似：

```
Program received signal SIGSEGV, Segmentation fault.
0x... in main () at null.c:4
4          *p = 0;
```

即定位到 `*p = 0;` 这条解引用空指针的语句。

3. valgrind --leak-check=yes ./null：memcheck 报告对地址 0 的非法写：

```
Invalid write of size 4
    at 0x...: main (null.c:4)
Address 0x0 is not stack'd, malloc'd or (recently) free'd
...
Process terminating with default action of signal 11 (SIGSEGV)
```

说明程序向既不在栈、也不是 malloc/free 过的非法地址 (0) 写数据，随后被 SIGSEGV 终止。

4. malloc 后忘记 free：

```
int *p = malloc(100 * sizeof(int));
/* 没有 free(p) */
return 0;
```

程序能正常运行并退出（进程结束时操作系统回收全部内存，不会报错）。gdb 看不出问题。valgrind --leak-check=yes 会报内存泄漏：

```
definitely lost: 400 bytes in 1 blocks
```

5. data 数组大小 100（下标 0..99），却写 data[100]：

```
int *data = malloc(100 * sizeof(int));
data[100] = 0;          /* 越界一个 int: heap buffer overflow */
```

直接运行常常不崩溃（越界一点点属于未定义行为，可能“看起来正常”）。valgrind 报堆越界写：

```
Invalid write of size 4
Address 0x... is 0 bytes after a block of size 400 alloc'd
```

程序是不正确的（写到了分配区之外）。

6. 分配、free，再访问其中元素：use-after-free。

```
int *data = malloc(100 * sizeof(int));
free(data);
printf("%d\n", data[0]); /* 访问已释放内存 */
```

程序通常能跑完并打印出一个值（可能是旧值或垃圾值，行为未定义）。valgrind 报对已释放块的非法读：

```
Invalid read of size 4
Address 0x... is 0 bytes inside a block of size 400 free'd
```

7. 把一个“中间地址”传给 free：

```
int *data = malloc(100 * sizeof(int));
free(data + 50); /* 不是 malloc 返回的指针 */
```

glibc 会检测到非法指针并 abort：free(): invalid pointer，进程收到 SIGABRT。也就是说不一定要靠工具——glibc 自身就能发现并终止；但 valgrind 能更精确地指出是非法 free：Invalid free() / delete / delete[]，并给出该地址相对某分配块的偏移。

8. 用 realloc 实现一个可增长的 vector，并与链表比较：

```
typedef struct { int *a; int len, cap; } vec_t;
void vec_push(vec_t *v, int x) {
    if (v->len == v->cap) {
        v->cap = v->cap ? v->cap * 2 : 1;
        v->a = realloc(v->a, v->cap * sizeof(int)); /* 翻倍扩容 */
    }
    v->a[v->len++] = x;
}
```

性能与对比：

- realloc 数组（翻倍扩容）：追加的摊还复杂度 $O(1)$ ，元素连续存放，随机访问 $O(1)$ 、cache 局部性好；缺点是扩容时可能整体拷贝（单次最坏 $O(n)$ ），可能产生内存碎片。若每次只 +1 扩容则退化为 $O(n^2)$ 拷贝。
- 链表：追加 $O(1)$ 、不需要拷贝/扩容；缺点是每个节点有指针开销，节点分散导致 cache 局部性差，随机访问 $O(n)$ 。
- 对“频繁追加 + 随机访问”的场景，翻倍扩容的 vector 通常整体更快。
- valgrind 可帮助发现这里常见的 bug：扩容失败未处理、忘记 free 旧缓冲、链表节点丢失导致的泄漏等。